```
/// <reference types="svelte" />
import App from './App.svelte';
import { mount } from 'svelte';

const app = mount(App, { target: document.getElementById('app')! });

export default app;
// ====== 导航卡片 ======
export interface DialItem {
  id: string;
  title: string;
  url: string;
  icon: string;
  groupId: string;
  sortOrder: number;
  createdAt: number;
}

// ====== 分组 ======
export interface DialGroup {
  id: string;
  name: string;
  sortOrder: number;
  isCollapsed: boolean;
}

// ====== 搜索引擎 ======
export interface SearchEngine {
  id: string;
  name: string;
  url: string;
  icon?: string;
  isCustom?: boolean;  // Pro 用户自定义引擎
  proOnly?: boolean;   // 仅 Pro 可用
}

export const FREE_ENGINE_LIMIT = 6;

// ====== 壁纸配置 ======
export type WallpaperType = 'solid' | 'gradient' | 'image' | 'builtin';

export interface WallpaperConfig {
  type: WallpaperType;
  value: string;
  blur: number;
  brightness: number;
}

// ====== 主题配置 ======
export type ThemeMode = 'light' | 'dark';
```

```
51
52  export interface ThemeConfig {
53    mode: ThemeMode;
54    primaryColor: string;
55    wallpaper: WallpaperConfig;
56  }
57
58  // ====== 时钟样式 ======
59  export type ClockStyle = 'digital' | 'minimal' | 'classic' | 'flip' | 'neon' | 'binary';
60
61  // ====== 全局设置 ======
62  export interface AppSettings {
63    searchEngine: string;
64    clockStyle: ClockStyle;
65    showDate: boolean;
66    showWeekday: boolean;
67    showRecentSites: boolean;
68    recentSitesCount: number;
69    openInNewTab: boolean;
70  }
71
72  // ====== 最近访问 ======
73  export interface RecentSite {
74    url: string;
75    title: string;
76    lastVisit: number;
77  }
78
79  // ====== 根数据 ======
80  export interface AppData {
81    version: number;
82    dials: DialItem[];
83    groups: DialGroup[];
84    searchEngines: SearchEngine[];
85    theme: ThemeConfig;
86    settings: AppSettings;
87    recentSites: RecentSite[];
88    customCss?: string;
89  }
90
91  // ====== 预设默认值 ======
92  export const DEFAULT_SEARCH_ENGINES: SearchEngine[] = [
93    { id: 'google', name: 'Google', url: 'https://www.google.com/search?q={keyword}', icon: '' },
94    { id: 'baidu', name: '百度', url: 'https://www.baidu.com/s?wd={keyword}', icon: '' },
95    { id: 'bing', name: 'Bing', url: 'https://www.bing.com/search?q={keyword}', icon: '' },
96    { id: 'sogou', name: '搜狗', url: 'https://www.sogou.com/web?query={keyword}', icon: '' },
97    { id: 'so', name: '360搜索', url: 'https://www.so.com/s?q={keyword}', icon: '' },
98    { id: 'zhihu', name: '知乎', url: 'https://www.zhihu.com/search?type=content&q={keyword}', icon: '' },
99    { id: 'weibo', name: '微博', url: 'https://s.weibo.com/weibo/{keyword}', icon: '', pro0nly: true },
100   { id: 'bilibili', name: '哔哩哔哩', url: 'https://search.bilibili.com/all?keyword={keyword}', icon: '', pro0nly: true },
```

```
  { id: 'github', name: 'GitHub', url: 'https://github.com/search?q={keyword}', icon: '', proOnly: true },
  { id: 'duckduckgo', name: 'DuckDuckGo', url: 'https://duckduckgo.com/?q={keyword}', icon: '', proOnly: true },
  { id: 'twitter', name: 'X (Twitter)', url: 'https://twitter.com/search?q={keyword}', icon: '', proOnly: true },
  { id: 'youtube', name: 'YouTube', url: 'https://www.youtube.com/results?search_query={keyword}', icon: '', proOnly: true
];

export const DEFAULT_SETTINGS: AppSettings = {
  searchEngine: 'google',
  clockStyle: 'digital',
  showDate: true,
  showWeekday: true,
  showRecentSites: true,
  recentSitesCount: 8,
  openInNewTab: true,
};

export const DEFAULT_THEME: ThemeConfig = {
  mode: 'light',
  primaryColor: '#3b82f6',
  wallpaper: { type: 'solid', value: '#f8fafc', blur: 0, brightness: 100 },
};

export const DEFAULT_WALLPAPERS: { id: string; name: string; value: string }[] = [
  { id: 'light-blue', name: '淡蓝', value: '#f0f4ff' },
  { id: 'light-green', name: '淡绿', value: '#f0fdf4' },
  { id: 'light-purple', name: '淡紫', value: '#faf5ff' },
  { id: 'light-gray', name: '淡灰', value: '#f8fafc' },
  { id: 'dark-blue', name: '深蓝', value: '#0f172a' },
  { id: 'dark-gray', name: '深灰', value: '#1e293b' },
  { id: 'dark-slate', name: '深岩', value: '#0f0f0f' },
  { id: 'gradient-sunset', name: '日落渐变', value: 'linear-gradient(135deg, #f97316, #ec4899)' },
  { id: 'gradient-ocean', name: '海洋渐变', value: 'linear-gradient(135deg, #06b6d4, #3b82f6)' },
  { id: 'gradient-forest', name: '森林渐变', value: 'linear-gradient(135deg, #22c55e, #06b6d4)' },
  { id: 'gradient-midnight', name: '午夜渐变', value: 'linear-gradient(135deg, #1e293b, #0f172a)' },
  { id: 'gradient-aurora', name: '极光渐变', value: 'linear-gradient(135deg, #3b82f6, #06b6d4)' },
];

export function generateId(): string {
  return Date.now().toString(36) + Math.random().toString(36).slice(2, 8);
}

// ====== 订阅 ======
export interface SubscriptionState {
  isPro: boolean;
  customEngines: SearchEngine[];
}
const STORAGE_KEY = 'speed-dial-data';
const CHUNK_PREFIX = 'speed-dial-chunk-';
const CHUNK_SIZE = 1024 * 1024; // 1MB per chunk

```

```
151 /**
152  * 加载数据：兼容旧格式（单key）和新格式（分片）
153  */
154 export function loadData<T>(): T | null {
155   try {
156     // 尝试分片格式
157     const chunkCount = parseInt(localStorage.getItem('speed-dial-chunks') || '0', 10);
158     if (chunkCount > 0) {
159       let raw = '';
160       for (let i = 0; i < chunkCount; i++) {
161         const chunk = localStorage.getItem(CHUNK_PREFIX + i);
162         if (chunk === null) {
163           console.warn('[Storage] 分片缺失:', i);
164           // 尝试降级到单key格式
165           return loadLegacy<T>();
166         }
167         raw += chunk;
168       }
169       return JSON.parse(raw) as T;
170     }
171     // 兼容旧格式
172     return loadLegacy<T>();
173   } catch (e) {
174     console.warn('[Storage] 数据解析失败', e);
175     return null;
176   }
177 }
178
179 function loadLegacy<T>(): T | null {
180   const raw = localStorage.getItem(STORAGE_KEY);
181   if (!raw) return null;
182   return JSON.parse(raw) as T;
183 }
184
185 /**
186  * 保存数据：小数据单key，大数据自动分片
187  */
188 export function saveData<T>(data: T): boolean {
189   try {
190     const serialized = JSON.stringify(data);
191     const totalSize = new Blob([serialized]).size;
192
193     // 清除旧数据
194     localStorage.removeItem(STORAGE_KEY);
195     clearChunks();
196
197     if (totalSize <= CHUNK_SIZE) {
198       // 小数据：单key存储
199       localStorage.setItem(STORAGE_KEY, serialized);
200       localStorage.removeItem('speed-dial-chunks');
```

```
    } else {
    // 大数据：分片存储
      const chunkCount = Math.ceil(totalSize / CHUNK_SIZE);
      for (let i = 0; i < chunkCount; i++) {
        const chunk = serialized.slice(i * CHUNK_SIZE, (i + 1) * CHUNK_SIZE);
        localStorage.setItem(CHUNK_PREFIX + i, chunk);
      }
      localStorage.setItem('speed-dial-chunks', String(chunkCount));
    }
    return true;
  } catch (e) {
    if (e instanceof DOMException && e.name === 'QuotaExceededError') {
    alert('存储空间不足！请减少自定义壁纸或清理数据后重试。');
    } else {
    console.error('[Storage] 保存失败', e);
    }
    return false;
  }
}

function clearChunks() {
  const count = parseInt(localStorage.getItem('speed-dial-chunks') || '0', 10);
  for (let i = 0; i < count; i++) {
    localStorage.removeItem(CHUNK_PREFIX + i);
  }
  localStorage.removeItem('speed-dial-chunks');
}

export function removeData(): void {
  try {
    localStorage.removeItem(STORAGE_KEY);
    clearChunks();
  } catch (e) {
    console.error('[Storage] 清除失败', e);
  }
}

export function checkStorageSupport(): boolean {
  try {
    const testKey = '__storage_test__';
    localStorage.setItem(testKey, '1');
    localStorage.removeItem(testKey);
    return true;
  } catch {
    return false;
  }
}

export function loadJsonFile<T>(file: File): Promise<T> {
  return new Promise((resolve, reject) => {
```

```
    const reader = new FileReader();
    reader.onload = () => {
      try {
        const data = JSON.parse(reader.result as string) as T;
        resolve(data);
      } catch {
       reject(new Error('JSON 格式错误'));
      }
    };
    reader.onerror = () => reject(new Error('文件读取失败'));
    reader.readAsText(file);
  });
}
import type { SearchEngine } from '../types';
import { DEFAULT_SEARCH_ENGINES, FREE_ENGINE_LIMIT } from '../types';
import { getIsPro, getCustomEngines } from '../stores/subscription.svelte';

export function getEngineById(id: string): SearchEngine | undefined {
  const all = getAllEngines();
  return all.find(e => e.id === id);
}

export function buildSearchUrl(engineId: string, keyword: string): string {
  const engine = getEngineById(engineId);
  if (!engine) {
    return `https://www.google.com/search?q=${encodeURIComponent(keyword)}`;
  }
  return engine.url.replace('{keyword}', encodeURIComponent(keyword));
}

/** 可用引擎（免费6种，Pro全部+自定义） */
export function getAvailableEngines(): SearchEngine[] {
  const pro = getIsPro();
  const builtin = pro
    ? DEFAULT_SEARCH_ENGINES
    : DEFAULT_SEARCH_ENGINES.filter(e => !e.proOnly).slice(0, FREE_ENGINE_LIMIT);

  if (pro) {
    const custom = getCustomEngines();
    if (custom.length > 0) return [...builtin, ...custom];
  }
  return builtin;
}

/** 全部引擎（含 Pro 专属，用于展示锁定状态） */
export function getAllEngines(): SearchEngine[] {
  const pro = getIsPro();
  const builtin = pro
    ? DEFAULT_SEARCH_ENGINES
    : DEFAULT_SEARCH_ENGINES.filter(e => !e.proOnly);
```

```
301
302   if (pro) {
303     const custom = getCustomEngines();
304     if (custom.length > 0) return [...builtin, ...custom];
305   }
306   return builtin;
307 }
308
309 /** 被锁定的引擎列表 */
310 export function getLockedEngines(): SearchEngine[] {
311   if (getIsPro()) return [];
312   return DEFAULT_SEARCH_ENGINES.filter(e => e.proOnly);
313 }
314 import type { ThemeMode, WallpaperConfig } from '../types';
315 import { getLang } from './i18n.svelte';
316
317 export function applyTheme(mode: ThemeMode, primaryColor: string): void {
318   const root = document.documentElement;
319   root.setAttribute('data-theme', mode);
320   root.style.setProperty('--primary-color', primaryColor);
321
322   const bgColor = mode === 'dark' ? '#0f172a' : '#f8fafc';
323   const textColor = mode === 'dark' ? '#e2e8f0' : '#1e293b';
324   const cardBg = mode === 'dark' ? 'rgba(30, 41, 59, 0.9)' : 'rgba(255, 255, 255, 0.85)';
325   const cardBorder = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.06)';
326   const hoverBg = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.04)';
327   const inputBg = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.02)';
328
329   root.style.setProperty('--bg-color', bgColor);
330   root.style.setProperty('--text-color', textColor);
331   root.style.setProperty('--card-bg', cardBg);
332   root.style.setProperty('--card-border', cardBorder);
333   root.style.setProperty('--hover-bg', hoverBg);
334   root.style.setProperty('--input-bg', inputBg);
335   root.style.setProperty('--color-scheme', mode === 'dark' ? 'dark' : 'light');
336
337   // 直接设置 body 背景色，确保暗色模式生效
338   document.body.style.backgroundColor = bgColor;
339   document.body.style.color = textColor;
340 }
341
342 export function applyWallpaper(wallpaper: WallpaperConfig): void {
343   const root = document.documentElement;
344   const { type, value, blur, brightness } = wallpaper;
345
346   if (type === 'solid') {
347     root.style.setProperty('--wallpaper', value);
348     root.style.setProperty('--wallpaper-filter', 'none');
349     root.style.setProperty('--bg-color', value);
350     adaptTextColor(value);
```

```
  } else if (type === 'gradient' || value.startsWith('linear-gradient') || value.startsWith('radial-gradient')) {
    root.style.setProperty('--wallpaper', value);
    root.style.setProperty('--wallpaper-filter', 'none');
    root.style.setProperty('--bg-color', value);
    adaptTextColor(value);
  } else if (type === 'image' || type === 'builtin') {
    root.style.setProperty('--wallpaper', `url(${value})`);
    root.style.setProperty('--wallpaper-filter', `blur(${blur}px) brightness(${brightness}%)`);
    root.style.setProperty('--bg-color', 'transparent');
  }
}

/** 从 CSS 值中提取第一个 hex 颜色，判断亮度并调整全部主题变量 */
function adaptTextColor(cssValue: string) {
  const match = cssValue.match(/#([0-9a-fA-F]{6})/);
  if (!match) return;
  const hex = match[1];
  const r = parseInt(hex.slice(0, 2), 16);
  const g = parseInt(hex.slice(2, 4), 16);
  const b = parseInt(hex.slice(4, 6), 16);
  const lum = (0.299 * r + 0.587 * g + 0.114 * b) / 255;
  const root = document.documentElement;

  if (lum < 0.5) {
    root.setAttribute('data-theme', 'dark');
    root.style.setProperty('--text-color', '#e2e8f0');
    root.style.setProperty('--card-bg', 'rgba(30, 41, 59, 0.9)');
    root.style.setProperty('--card-border', 'rgba(255,255,255,0.06)');
    root.style.setProperty('--hover-bg', 'rgba(255,255,255,0.06)');
    root.style.setProperty('--input-bg', 'rgba(255,255,255,0.06)');
    document.body.style.color = '#e2e8f0';
  } else {
    root.setAttribute('data-theme', 'light');
    root.style.setProperty('--text-color', '#1e293b');
    root.style.setProperty('--card-bg', 'rgba(255, 255, 255, 0.85)');
    root.style.setProperty('--card-border', 'rgba(0,0,0,0.06)');
    root.style.setProperty('--hover-bg', 'rgba(0,0,0,0.04)');
    root.style.setProperty('--input-bg', 'rgba(0,0,0,0.02)');
    document.body.style.color = '#1e293b';
  }
}

export function formatDate(date: Date): string {
  const lang = getLang();
  if (lang === 'zh-CN') {
    const y = date.getFullYear();
    const m = String(date.getMonth() + 1).padStart(2, '0');
    const d = String(date.getDate()).padStart(2, '0');
    return `${y}年${m}月${d}日`;
  }
```

```
  return date.toLocaleDateString('en-US', { year: 'numeric', month: 'short', day: 'numeric' });
}

export function formatWeekday(date: Date): string {
  const lang = getLang();
  const weekdays: Record<string, string[]> = {
  'zh-CN': ['星期日', '星期一', '星期二', '星期三', '星期四', '星期五', '星期六'],
    'en': ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'],
  };
  return (weekdays[lang] || weekdays['zh-CN'])[date.getDay()];
}
/**
* 键盘快捷键管理器
* 仅在无 input/textarea 聚焦时触发
 */
type ShortcutHandler = (e: KeyboardEvent) => void;

interface Shortcut {
  key: string;
  ctrl?: boolean;
  shift?: boolean;
  alt?: boolean;
  handler: ShortcutHandler;
  description: string;
}

const shortcuts: Shortcut[] = [];
let registered = false;

function isEditing(): boolean {
  const el = document.activeElement;
  if (!el) return false;
  const tag = el.tagName.toLowerCase();
  if (tag === 'input' || tag === 'textarea' || tag === 'select') return true;
  if ((el as HTMLElement).isContentEditable) return true;
  return false;
}

function handleKeydown(e: KeyboardEvent) {
  if (isEditing()) return;

  for (const s of shortcuts) {
    const ctrlOk = s.ctrl ? (e.ctrlKey || e.metaKey) : true;
    const shiftOk = s.shift ? e.shiftKey : true;
    const altOk = s.alt ? e.altKey : true;

    if (e.key === s.key && ctrlOk && shiftOk && altOk) {
      e.preventDefault();
      s.handler(e);
      return;
```

```
    }
  }
}

export function registerShortcut(
  key: string,
  handler: ShortcutHandler,
  description: string,
  modifiers: { ctrl?: boolean; shift?: boolean; alt?: boolean } = {}
) {
  shortcuts.push({ key, handler, description, ...modifiers });

  if (!registered) {
    document.addEventListener('keydown', handleKeydown);
    registered = true;
  }
}

/**
 * 获取所有快捷键列表（用于帮助面板）
 */
export function getShortcuts(): { key: string; description: string }[] {
  return shortcuts.map(s => ({
    key: [
      s.ctrl ? 'Ctrl+' : '',
      s.shift ? 'Shift+' : '',
      s.alt ? 'Alt+' : '',
      s.key,
    ].join(''),
    description: s.description,
  }));
}

/**
 * 聚焦搜索框
 */
export function focusSearch() {
  const el = document.querySelector<HTMLInputElement>('.search-input');
  if (el) {
    el.focus();
    el.select();
  }
}
export interface WeatherData {
  location: {
    city: string;
    province: string;
  };
  current: {
    temperature: number;
```

```
    humidity: number;
    weather: string;
    windDirection: string;
    windScale: string;
    icon: string;
  };
  forecast: Array<{
    date: string;
    dayWeather: string;
    dayTemp: string;
    nightTemp: string;
    dayWeatherIcon: string;
  }>;
  updateTime: string;
}

interface ApiResponse {
  code: number;
  msg: string;
  data: {
    data: {
      location: {
        city: string;
        province: string;
      };
      current: {
        temperature: number;
        humidity: number;
        windDirection: string;
        windScale: string;
      };
      forecast: Array<{
        date: string;
        dayWeather: string;
        dayTemp: string;
        nightTemp: string;
        dayWeatherIcon: string;
      }>;
    };
  };
}

const API_URL = 'https://api.ruseo.cn/api/weather_ip';
const API_KEY = 'be4385f8-e5a2-49a4-b3ad-c6f23d1f38a1';
const CACHE_KEY = 'speed-dial-weather';
const CACHE_DURATION = 15 * 60 * 1000; // 15分钟缓存

export async function fetchRandomWallpaper(): Promise<string | null> {
  try {
    const response = await fetch(`https://api.ruseo.cn/api/wallpaper?key=${API_KEY}`);
```

```
    const result = await response.json();

    if (result.code === 0 && result.data?.image_url) {
    // 验证图片 URL 可访问，xxapi.cn 可能 HTTP/2 异常
      try {
        const check = await fetch(result.data.image_url, { method: 'HEAD', mode: 'no-cors' });
        if (check.ok || check.type === 'opaque') {
          return result.data.image_url;
        }
    } catch { /* 图片不可访问，使用后备 */ }
    }

  // 后备：使用 Picsum 随机壁纸（全球 CDN，可靠）
    const seed = Date.now();
    return `https://picsum.photos/seed/${seed}/1920/1080`;
  } catch {
  // API 完全不可用，直接后备
    const seed = Date.now();
    return `https://picsum.photos/seed/${seed}/1920/1080`;
  }
}

export interface LunarData {
  date: string;
  weekday: string;
  lunar: {
    formatted: string;
    year_ganzhi: string;
    year_shengxiao: string;
    day_ganzhi: string;
    month_ganzhi: string;
  };
  zodiac: string;
  solar_term: {
    current: string | null;
    prev: string;
    next: string;
    next_date: string;
  };
  holiday: {
    today: Array<{ name: string; type?: string }>;
  };
  almanac: {
    yi: string[];
    ji: string[];
    xi_shen: string;
    fu_shen: string;
    cai_shen: string;
  };
}
```

```
601
602 const LUNAR_CACHE_KEY = 'speed-dial-lunar';
603 const LUNAR_CACHE_DURATION = 1 * 60 * 60 * 1000; // 1小时缓存
604
605 export async function fetchLunar(date?: string): Promise<LunarData | null> {
606   try {
607     const cacheKey = date ? `${LUNAR_CACHE_KEY}-${date}` : LUNAR_CACHE_KEY;
608
609     const cached = getCachedLunar(cacheKey);
610     if (cached) {
611       return cached;
612     }
613
614     const url = date
615       ? `https://api.ruseo.cn/api/lunar?key=${API_KEY}&date=${date}&scope=all`
616       : `https://api.ruseo.cn/api/lunar?key=${API_KEY}&scope=all`;
617
618     const response = await fetch(url);
619     const result = await response.json();
620
621     if (result.code !== 0 || !result.data) {
622       console.error('Lunar API error:', result.msg);
623       return null;
624     }
625
626     const data: LunarData = {
627       date: result.data.date,
628       weekday: result.data.weekday,
629       lunar: {
630         formatted: result.data.lunar?.formatted || '',
631         year_ganzhi: result.data.lunar?.year_ganzhi || '',
632         year_shengxiao: result.data.lunar?.year_shengxiao || '',
633         day_ganzhi: result.data.lunar?.day_ganzhi || '',
634         month_ganzhi: result.data.lunar?.month_ganzhi || '',
635       },
636       zodiac: result.data.zodiac || '',
637       solar_term: {
638         current: result.data.solar_term?.current || null,
639         prev: result.data.solar_term?.prev || '',
640         next: result.data.solar_term?.next || '',
641         next_date: result.data.solar_term?.next_date || '',
642       },
643       holiday: {
644         today: result.data.holiday?.today || [],
645       },
646       almanac: {
647         yi: result.data.almanac?.yi || [],
648         ji: result.data.almanac?.ji || [],
649         xi_shen: result.data.almanac?.xi_shen || '',
650         fu_shen: result.data.almanac?.fu_shen || '',
```

```
        cai_shen: result.data.almanac?.cai_shen || '',
      },
    };

    cacheLunar(cacheKey, data);
    return data;
  } catch (error) {
    console.error('Failed to fetch lunar:', error);
    return null;
  }
}

function getCachedLunar(cacheKey: string): LunarData | null {
  try {
    const cached = localStorage.getItem(cacheKey);
    if (!cached) return null;

    const { data, timestamp } = JSON.parse(cached);
    if (Date.now() - timestamp > LUNAR_CACHE_DURATION) {
      localStorage.removeItem(cacheKey);
      return null;
    }

    return data;
  } catch {
    return null;
  }
}

function cacheLunar(cacheKey: string, data: LunarData): void {
  try {
    localStorage.setItem(cacheKey, JSON.stringify({
      data,
      timestamp: Date.now(),
    }));
  } catch {
  }
}

export async function fetchWeather(): Promise<WeatherData | null> {
  try {
  // 检查缓存
    const cached = getCachedWeather();
    if (cached) {
      return cached;
    }

    const response = await fetch(`${API_URL}?key=${API_KEY}&day=3&hourtype=0&suntimetype=0`);
    const result: ApiResponse = await response.json();

```

```
    if (result.code !== 0 || !result.data?.data) {
      console.error('Weather API error:', result.msg);
      return null;
    }

    const apiData = result.data.data;
    const data: WeatherData = {
      location: {
        city: apiData.location.city,
        province: apiData.location.province,
      },
      current: {
        temperature: apiData.current.temperature,
        humidity: apiData.current.humidity,
        weather: apiData.forecast[0]?.dayWeather || '',
        windDirection: apiData.current.windDirection,
        windScale: apiData.current.windScale,
        icon: apiData.forecast[0]?.dayWeatherIcon || '',
      },
      forecast: apiData.forecast.slice(0, 3),
      updateTime: new Date().toLocaleString('zh-CN'),
    };

    // 缓存结果
    cacheWeather(data);
    return data;
  } catch (error) {
    console.error('Failed to fetch weather:', error);
    return null;
  }
}

function getCachedWeather(): WeatherData | null {
  try {
    const cached = localStorage.getItem(CACHE_KEY);
    if (!cached) return null;

    const { data, timestamp } = JSON.parse(cached);
    if (Date.now() - timestamp > CACHE_DURATION) {
      localStorage.removeItem(CACHE_KEY);
      return null;
    }

    return data;
  } catch {
    return null;
  }
}

function cacheWeather(data: WeatherData): void {
```

```
  try {
    localStorage.setItem(CACHE_KEY, JSON.stringify({
      data,
      timestamp: Date.now(),
    }));
  } catch {
  // 忽略存储错误
  }
}
export interface BookmarkItem {
  title: string;
  url: string;
}

export interface BookmarkGroup {
  name: string;
  items: BookmarkItem[];
}

/**
 * 解析浏览器导出的书签 HTML，按文件夹分组
 */
export function parseBookmarkGroups(html: string): BookmarkGroup[] {
  const parser = new DOMParser();
  const doc = parser.parseFromString(html, 'text/html');
  const groups: BookmarkGroup[] = [];
  const seen = new Set<string>();

  // 遍历所有 DT 节点
  const dtNodes = doc.querySelectorAll('dt');
  let currentGroup: BookmarkGroup = { name: '默认收藏', items: [] };

  dtNodes.forEach(dt => {
    const h3 = dt.querySelector(':scope > h3');
    const a = dt.querySelector(':scope > a[href]');

    if (h3) {
    // 新文件夹
    const name = h3.textContent?.trim() || '未命名';
    // 保存上一个分组（如果有内容）
      if (currentGroup.items.length > 0) {
        groups.push(currentGroup);
      }
      currentGroup = { name, items: [] };
    } else if (a) {
      const url = a.getAttribute('href')?.trim() || '';
      const title = a.textContent?.trim() || '';
      if (!url || url.startsWith('javascript:') || url.startsWith('about:')) return;
      if (!title) return;
      const key = url.toLowerCase();
```

```
      if (seen.has(key)) return;
      seen.add(key);
      currentGroup.items.push({ title, url });
    }
  });

  // 最后一个分组
  if (currentGroup.items.length > 0) {
    groups.push(currentGroup);
  }

  return groups;
}
import type { AppData } from '../types';
import { getDialsState } from '../stores/dials.svelte';
import { getTheme } from '../stores/theme.svelte';
import { getSettings } from '../stores/settings.svelte';
import { getRecentSites } from '../stores/recentSites.svelte';

const API_BASE = 'https://sync.ruseo.cn/api/sync.php';

function getToken(): string | null {
  return localStorage.getItem('quick-dial-token');
}

function setToken(token: string) {
  localStorage.setItem('quick-dial-token', token);
}

function getLocalVersion(): number {
  return parseInt(localStorage.getItem('quick-dial-version') || '0', 10);
}

function setLocalVersion(v: number) {
  localStorage.setItem('quick-dial-version', String(v));
}

export function getLastSyncTime(): string | null {
  return localStorage.getItem('quick-dial-last-sync');
}

function setLastSyncTime() {
  localStorage.setItem('quick-dial-last-sync', new Date().toISOString());
}

export function isLoggedIn(): boolean {
  return !!getToken();
}

export function getUsername(): string | null {
```

```
  return localStorage.getItem('quick-dial-username');
}

// ====== 认证 ======

export async function register(username: string, password: string): Promise<{ ok: boolean; msg: string }> {
  try {
    const res = await fetch(`${API_BASE}?action=register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password }),
    });
    const result = await res.json();
    if (result.code === 201) {
      setToken(result.data.token);
      localStorage.setItem('quick-dial-username', result.data.username);
      return { ok: true, msg: '注册成功' };
    }
    return { ok: false, msg: result.msg || '注册失败' };
  } catch {
    return { ok: false, msg: '网络错误' };
  }
}

export async function login(username: string, password: string): Promise<{ ok: boolean; msg: string }> {
  try {
    const res = await fetch(`${API_BASE}?action=login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password }),
    });
    const result = await res.json();
    if (result.code === 200) {
      setToken(result.data.token);
      localStorage.setItem('quick-dial-username', result.data.username);
      return { ok: true, msg: '登录成功' };
    }
    return { ok: false, msg: result.msg || '登录失败' };
  } catch {
    return { ok: false, msg: '网络错误' };
  }
}

export function logout() {
  localStorage.removeItem('quick-dial-token');
  localStorage.removeItem('quick-dial-username');
}

// ====== 获取当前本地数据快照 ======

```

```
function getLocalSnapshot(): AppData {
  const dialsState = getDialsState();
  return {
    version: 1,
    dials: dialsState.items,
    groups: dialsState.groups,
    searchEngines: [],
    theme: getTheme(),
    settings: getSettings(),
    recentSites: getRecentSites().map(s => ({ ...s })),
  };
}

// ====== 上传 ======

export async function uploadSync(): Promise<{ ok: boolean; msg: string }> {
  const token = getToken();
  if (!token) return { ok: false, msg: '未登录' };

  const data = getLocalSnapshot();
  const localVersion = getLocalVersion();

  try {
    const res = await fetch(`${API_BASE}?action=upload`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`,
      },
      body: JSON.stringify({ data, version: localVersion }),
    });
    const result = await res.json();

    if (result.code === 200) {
      setLocalVersion(result.data.version);
      setLastSyncTime();
      return { ok: true, msg: '同步成功' };
    }
    if (result.code === 409) {
    // 云端版本更新，自动下载
      const dl = await downloadSync();
      if (dl.ok) {
        setLocalVersion(result.data.server_version);
        setLastSyncTime();
       return { ok: true, msg: '已合并云端数据' };
      }
    return { ok: false, msg: '服务器数据更新，请手动下载后再上传' };
    }
   return { ok: false, msg: result.msg || '上传失败' };
  } catch {
```

```
    return { ok: false, msg: '网络错误' };
  }
}

// ====== 下载 ======

export async function downloadSync(): Promise<{ ok: boolean; msg: string; data?: AppData }> {
  const token = getToken();
  if (!token) return { ok: false, msg: '未登录' };

  try {
    const res = await fetch(`${API_BASE}?action=download`, {
      method: 'GET',
      headers: { 'Authorization': `Bearer ${token}` },
    });
    const result = await res.json();

    if (result.code === 200) {
      setLocalVersion(result.data.version);
      setLastSyncTime();
      return { ok: true, msg: '下载成功', data: result.data.data as AppData };
    }
    if (result.code === 404) {
      return { ok: false, msg: '云端暂无数据，请先上传' };
    }
    return { ok: false, msg: result.msg || '下载失败' };
  } catch {
    return { ok: false, msg: '网络错误' };
  }
}
/**
 * Quick Dial 支付 SDK
 * 与 sync.ruseo.cn/api/pay.php 通信
 */

const PAY_API = 'https://sync.ruseo.cn/api/pay.php';

function getToken(): string | null {
  return localStorage.getItem('quick-dial-token');
}

export interface PlanInfo {
  id: string;
  name: string;
  amount: number;
  days: number | null;
}

export const PLANS: PlanInfo[] = [
  { id: 'monthly', name: '月度', amount: 9.90, days: 30 },
```

```
  { id: 'yearly', name: '年度', amount: 68.00, days: 365 },
  { id: 'lifetime', name: '终身', amount: 198.00, days: null },
];

export interface OrderResult {
  ok: boolean;
  msg: string;
  qrCode?: string;     // 微信扫码链接
  payUrl?: string;     // 支付宝跳转链接
  orderNo?: string;
  plan?: string;
  amount?: number;
  method?: string;
}

export interface SubStatus {
  isPro: boolean;
  plan: string;
  expireAt: string | null;
}

/**
 * 创建支付订单
 */
export async function createOrder(
  plan: string,
  method: 'wechat' | 'alipay'
): Promise<OrderResult> {
  const token = getToken();
  if (!token) return { ok: false, msg: '请先登录云同步账号' };

  try {
    const res = await fetch(`${PAY_API}?action=create_order`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`,
      },
      body: JSON.stringify({ plan, method }),
    });
    const result = await res.json();

    if (result.code === 200) {
      return {
        ok: true,
        msg: '订单已创建',
        qrCode: result.data.qr_code || undefined,
        payUrl: result.data.pay_url || undefined,
        orderNo: result.data.order_no,
        plan: result.data.plan,
```

```
        amount: result.data.amount,
        method: result.data.method,
      };
    }
    return { ok: false, msg: result.msg || '创建订单失败' };
  } catch {
    return { ok: false, msg: '网络错误' };
  }
}

/**
 * 查询订阅状态
 */
export async function checkSubscription(): Promise<SubStatus> {
  const token = getToken();
  if (!token) return { isPro: false, plan: 'free', expireAt: null };

  try {
    const res = await fetch(`${PAY_API}?action=status`, {
      headers: { 'Authorization': `Bearer ${token}` },
    });
    const result = await res.json();

    if (result.code === 200) {
      return {
        isPro: result.data.is_pro,
        plan: result.data.plan,
        expireAt: result.data.expire_at,
      };
    }
    return { isPro: false, plan: 'free', expireAt: null };
  } catch {
    return { isPro: false, plan: 'free', expireAt: null };
  }
}

/**
 * 生成支付二维码 SVG（用于展示）
 */
export function generateQRCodeSVG(text: string, size: number = 200): string {
  // 使用在线 API 生成二维码
  return `https://api.qrserver.com/v1/create-qr-code/?size=${size}x${size}&data=${encodeURIComponent(text)}`;
}
type Lang = 'zh-CN' | 'en';
type Dict = Record<string, string>;

const zh: Dict = {
  'search.placeholder': '搜索或输入网址','search.go': '搜索','search.engine': '搜索引擎',
  'clock.styles': '时钟样式','clock.digital': '数字','clock.minimal': '极简','clock.classic': '经典','clock.flip': '翻页','clock.neon':
  'settings.title': '设置','settings.theme': '外观主题','settings.themeHint': '壁纸自动适配深浅模式','settings.showDate': '显示日期','settings.
```

```
  'pro.title': 'Quick Dial Pro','pro.active': '已激活','pro.inactive': '未激活','pro.upgrade': '升级 Quick Dial Pro','pro.renew': '
  'sync.title': '云同步','sync.manual': '数据不会自动同步，点按钮手动上传或下载','sync.upload': '上传到云端','sync.download': '从云端下载','sync.downloaded
  'ie.title': '导入 / 导出','ie.export': '导出备份','ie.import': '导入备份','ie.importBookmarks': '导入浏览器书签','ie.clear': '清空所有数据','ie.cl
  'dial.add': '添加导航','dial.edit': '编辑导航','dial.name': '网站名称','dial.url': '网站链接','dial.urlPlaceholder': '粘贴链接自动识别名称','dial.i
  'wp.title': '壁纸设置','wp.random': '随机壁纸','wp.upload': '上传图片',
  'sub.title': '升级 Quick Dial Pro','sub.pay': '立即支付','sub.creating': '创建订单中...','sub.scanHint': '请使用{method}扫码支付','sub.paid
  'stats.title': '访问统计','stats.total': '总点击','stats.sites': '站点数','stats.ranking': '点击排行','stats.empty': '暂无访问数据','stats.cl
  'onboard.skip': '跳过','onboard.next': '下一步','onboard.start': '开始使用','onboard.step1Title': '搜索直达','onboard.step1Desc': '在搜索
  'help.title': '键盘快捷键','help.tip': '按 ? 随时打开此面板',
  'footer.copyright': '呲啦起始页',
  'group.manage': '分组管理','group.default': '默认收藏','group.unnamed': '未命名',
  'common.close': '关闭','common.save': '保存','common.cancel': '取消','common.loading': '处理中...',
  'weather.humidity': '湿度','weather.wind': '风力','weather.forecast': '天气预报',
  'pay.wechat': '微信支付','pay.alipay': '支付宝','pay.monthly': '月度','pay.yearly': '年度','pay.lifetime': '终身',
    // Additional UI
  'recent.title': '最近访问','recent.empty': '暂无最近访问',
  'dial.empty': '点击 + 添加第一个导航',
  'group.limit': '{current}/{max} 个分组 • Pro 无限',
  'group.add': '添加分组','group.done': '完成','group.addBtn': '添加',
  'wp.local': '上传本地图片','wp.url': '输入图片链接','wp.apply': '应用','wp.placeholder': '输入图片链接...',
  'sync.never': '从未同步',
  'dial.addToGroup': '添加到本组',
  'pro.monthly': '月度会员','pro.yearly': '年度会员','pro.lifetime': '终身会员','pro.expire': '到期：',
  'settings.engine': '默认搜索引擎',
  'pro.customFooterEg': '例如：我的公司',
  'cat.common': '常用','cat.social': '社交','cat.dev': '开发','cat.media': '媒体','cat.office': '办公','cat.study': '学习','cat.brand':
  'pro.cssPlaceholder': '/* 在此输入自定义 CSS */',
  'icon.customUrl': '自定义图标 URL',
    // Default group names (stored in data, translated at render)
  '常用': '常用',
};

const en: Dict = {
  'search.placeholder': 'Search or enter URL','search.go': 'Search','search.engine': 'Search Engine',
  'clock.styles': 'Clock Style','clock.digital': 'Digital','clock.minimal': 'Minimal','clock.classic': 'Classic','clock.fli
  'settings.title': 'Settings','settings.theme': 'Appearance','settings.themeHint': 'Adapts to wallpaper','settings.showDat
  'pro.title': 'Quick Dial Pro','pro.active': 'Active','pro.inactive': 'Inactive','pro.upgrade': 'Upgrade to Pro','pro.rene
  'sync.title': 'Cloud Sync','sync.manual': 'Sync is manual - upload or download on demand','sync.upload': 'Upload','sync.d
  'ie.title': 'Import / Export','ie.export': 'Export','ie.import': 'Import','ie.importBookmarks': 'Import Bookmarks','ie.cl
  'dial.add': 'Add Dial','dial.edit': 'Edit Dial','dial.name': 'Site Name','dial.url': 'Site URL','dial.urlPlaceholder': 'P
  'wp.title': 'Wallpaper','wp.random': 'Random','wp.upload': 'Upload',
  'sub.title': 'Upgrade to Pro','sub.pay': 'Pay Now','sub.creating': 'Creating...','sub.scanHint': 'Scan with {method}','su
  'stats.title': 'Statistics','stats.total': 'Total Clicks','stats.sites': 'Sites','stats.ranking': 'Top Sites','stats.empt
  'onboard.skip': 'Skip','onboard.next': 'Next','onboard.start': 'Start','onboard.step1Title': 'Search','onboard.step1Desc'
  'help.title': 'Shortcuts','help.tip': 'Press ? anytime',
  'footer.copyright': 'Quick Dial',
  'group.manage': 'Groups','group.default': 'Default','group.unnamed': 'Unnamed',
  'common.close': 'Close','common.save': 'Save','common.cancel': 'Cancel','common.loading': 'Loading...',
  'weather.humidity': 'Humidity','weather.wind': 'Wind','weather.forecast': 'Forecast',
  'pay.wechat': 'WeChat','pay.alipay': 'Alipay','pay.monthly': 'Monthly','pay.yearly': 'Yearly','pay.lifetime': 'Lifetime',
```

```
    'recent.title': 'Recent Sites','recent.empty': 'No recent sites',
    'dial.empty': 'Click + to add your first dial',
    'group.limit': '{current}/{max} groups • Pro unlimited',
    'group.add': 'Add Group','group.done': 'Done','group.addBtn': 'Add',
    'wp.local': 'Upload Image','wp.url': 'Image URL','wp.apply': 'Apply','wp.placeholder': 'Paste image URL...',
    'sync.never': 'Never synced',
    'dial.addToGroup': 'Add to Group',
    'pro.monthly': 'Monthly','pro.yearly': 'Yearly','pro.lifetime': 'Lifetime','pro.expire': 'Expires: ',
    'settings.engine': 'Default Engine',
    'pro.customFooterEg': 'e.g. My Company',
    'cat.common': 'Common','cat.social': 'Social','cat.dev': 'Dev','cat.media': 'Media','cat.office': 'Office','cat.study': '
    'pro.cssPlaceholder': '/* Enter custom CSS here */',
    'icon.customUrl': 'Custom Icon URL',
    '常用': 'Default',
  };

  let currentLang = $state<Lang>((localStorage.getItem('qd-lang') as Lang) || 'zh-CN');
  let _v = $state(0); // 版本号，强制 Svelte 追踪变化

  export function t(key: string): string {
    void _v; // 建立响应式依赖
    const dict = currentLang === 'zh-CN' ? zh : en;
    return dict[key] || key;
  }

  export function getLang(): Lang {
    return currentLang;
  }

  export function setLang(lang: Lang) {
    currentLang = lang;
    _v++;
    localStorage.setItem('qd-lang', lang);
    document.documentElement.setAttribute('lang', lang);
    document.title = lang === 'zh-CN'
    ? '啦啦起始页 - 极简无广告浏览器新标签页'
      : 'Quick Dial - Clean, Ad-Free Browser New Tab';
  }

  // 初始化
  setLang(currentLang);
  /**
   * 右键菜单集成 — 监听 Chrome context menu 添加的页面
   */
  interface ContextAddData {
    title: string;
    url: string;
  }

  /**
```

```
 * 检查并获取右键菜单添加的页面数据（消费后自动清除）
 */
export async function getContextAdd(): Promise<ContextAddData | null> {
  if (typeof chrome === 'undefined' || !chrome.storage) return null;
  try {
    const result = await chrome.storage.local.get('qd-context-add');
    const data = result['qd-context-add'] as ContextAddData | undefined;
    if (data?.url) {
      // 消费后清除，防止重复添加
      await chrome.storage.local.remove('qd-context-add');
      return data;
    }
  } catch { /* 非扩展环境静默 */ }
  return null;
}
type ToastType = 'success' | 'error' | 'info';

interface ToastItem {
  id: number;
  message: string;
  type: ToastType;
}

let toasts = $state<ToastItem[]>([]);
let nextId = 0;

export function getToasts(): Readonly<ToastItem[]> {
  return toasts;
}

export function showToast(message: string, type: ToastType = 'info', duration = 2500) {
  const id = nextId++;
  toasts = [...toasts, { id, message, type }];

  if (duration > 0) {
    setTimeout(() => {
      toasts = toasts.filter(t => t.id !== id);
    }, duration);
  }
}

export function dismissToast(id: number) {
  toasts = toasts.filter(t => t.id !== id);
}
  import type { DialItem, DialGroup } from '../types';
  import { generateId } from '../types';
  import { t } from '../utils/i18n.svelte';
import { getIsPro } from './subscription.svelte';

export const FREE_GROUP_LIMIT = 3;
```

```

interface DialsState {
  items: DialItem[];
  groups: DialGroup[];
}

let state = $state<DialsState>({ items: [], groups: [] });

export function initDials(data: { dials: DialItem[]; groups: DialGroup[] }): void {
  state.items = data.dials || [];
  state.groups = data.groups || [];
}

/**
 * 获取状态快照，防止外部直接修改
 */
export function getDialsState(): Readonly<DialsState> {
  return {
    items: state.items.map(item => ({ ...item })),
    groups: state.groups.map(group => ({ ...group }))
  };
}

/**
 * 获取内部状态引用（仅限内部使用）
 */
export function getDialsStateInternal(): DialsState {
  return state;
}

// ====== 分组操作 ======
export function addGroup(name: string): DialGroup | null {
  if (!getIsPro() && state.groups.length >= FREE_GROUP_LIMIT) {
    return null;
  }
  const newGroup: DialGroup = {
    id: generateId(),
    name,
    sortOrder: state.groups.length,
    isCollapsed: false,
  };
  state.groups = [...state.groups, newGroup];
  return newGroup;
}

export function updateGroup(id: string, updates: Partial<DialGroup>): void {
  state.groups = state.groups.map(g => (g.id === id ? { ...g, ...updates } : g));
}

export function removeGroup(id: string): void {
```

```
  state.groups = state.groups.filter(g => g.id !== id);
  // 移除组内所有卡片
  state.items = state.items.filter(d => d.groupId !== id);
}

export function reorderGroups(groupIds: string[]): void {
  state.groups = groupIds.map((id, i) => {
    const g = state.groups.find(gg => gg.id === id);
    if (!g) {
      console.warn(`[reorderGroups] Group not found: ${id}`);
      return null;
    }
    return { ...g, sortOrder: i };
  }).filter((g): g is DialGroup => g !== null);
}

// ====== 卡片操作 ======
export function addDial(dial: Omit<DialItem, 'id' | 'createdAt'>): DialItem {
  const newDial: DialItem = {
    ...dial,
    id: generateId(),
    createdAt: Date.now(),
  };
  state.items = [...state.items, newDial];
  return newDial;
}

export function updateDial(id: string, updates: Partial<DialItem>): void {
  state.items = state.items.map(d => (d.id === id ? { ...d, ...updates } : d));
}

export function removeDial(id: string): void {
  state.items = state.items.filter(d => d.id !== id);
}

/**
 * 重新排序卡片，同时更新 sortOrder 字段
 */
export function reorderDials(dialIds: string[]): void {
  state.items = dialIds.map((id, i) => {
    const d = state.items.find(dd => dd.id === id);
    if (!d) {
      console.warn(`[reorderDials] Dial not found: ${id}`);
      return null;
    }
    return { ...d, sortOrder: i };
  }).filter((d): d is DialItem => d !== null);
}

export function moveDialToGroup(dialId: string, targetGroupId: string): void {
```

```
  const dial = state.items.find(d => d.id === dialId);
  if (!dial) return;

  // 移动到目标组末尾
  const groupItems = state.items.filter(d => d.groupId === targetGroupId);
  const maxSortOrder = groupItems.length > 0
    ? Math.max(...groupItems.map(d => d.sortOrder))
    : -1;

  state.items = state.items.map(d =>
    d.id === dialId ? { ...d, groupId: targetGroupId, sortOrder: maxSortOrder + 1 } : d
  );
}

// ====== 默认分组 ======
export function ensureDefaultGroup(): string {
  const defaultName = t('group.default');
  let defaultGroup = state.groups.find(g => g.name === defaultName || g.name === '常用');
  if (!defaultGroup) {
    defaultGroup = addGroup(defaultName);
  }
  return defaultGroup.id;
}
import type { ThemeConfig, ThemeMode, WallpaperConfig } from '../types';
import { DEFAULT_THEME } from '../types';
import { applyTheme, applyWallpaper } from '../utils/theme';

let theme = $state<ThemeConfig>({ ...DEFAULT_THEME });
let userChoseWallpaper = $state(false);

function isDefaultWallpaper(wp: WallpaperConfig): boolean {
  const d = DEFAULT_THEME.wallpaper;
  return wp.type === d.type && wp.value === d.value;
}

export function initTheme(data: ThemeConfig | undefined): void {
  theme = { ...DEFAULT_THEME, ...data };
  userChoseWallpaper = data?.wallpaper ? !isDefaultWallpaper(data.wallpaper) : false;

  applyTheme(theme.mode, theme.primaryColor);
  applyAdaptiveWallpaper(theme.mode);
}

export function getTheme(): ThemeConfig {
  return {
    mode: theme.mode,
    primaryColor: theme.primaryColor,
    wallpaper: { ...theme.wallpaper }
  };
}
```

```
1401
1402  export function setThemeMode(mode: ThemeMode): void {
1403    theme.mode = mode;
1404    applyTheme(mode, theme.primaryColor);
1405    applyAdaptiveWallpaper(mode);
1406  }
1407
1408  /** 自适应壁纸：默认壁纸跟随暗/亮模式，用户自选壁纸不干预 */
1409  function applyAdaptiveWallpaper(mode: ThemeMode) {
1410    if (!userChoseWallpaper && theme.wallpaper.type === 'solid') {
1411      const v = mode === 'dark' ? '#0f172a' : '#f8fafc';
1412      applyWallpaper({ type: 'solid', value: v, blur: 0, brightness: 100 });
1413    } else {
1414      applyWallpaper(theme.wallpaper);
1415    }
1416  }
1417
1418  export function setPrimaryColor(color: string): void {
1419    theme.primaryColor = color;
1420    applyTheme(theme.mode, color);
1421  }
1422
1423  export function setWallpaper(wallpaper: WallpaperConfig): void {
1424    theme.wallpaper = { ...wallpaper };
1425    userChoseWallpaper = true;
1426    applyWallpaper(wallpaper);
1427  }
1428
1429  export function getWallpaperConfig(): WallpaperConfig {
1430    return { ...theme.wallpaper };
1431  }
1432  import type { AppSettings, ClockStyle } from '../types';
1433  import { DEFAULT_SETTINGS } from '../types';
1434
1435  let settings = $state<AppSettings>({ ...DEFAULT_SETTINGS });
1436
1437  export function initSettings(data: AppSettings | undefined): void {
1438    settings = { ...DEFAULT_SETTINGS, ...data };
1439  }
1440
1441  /**
1442   * 获取设置快照，防止外部直接修改
1443   */
1444  export function getSettings(): Readonly<AppSettings> {
1445    return { ...settings };
1446  }
1447
1448  export function setSearchEngine(id: string): void {
1449    settings.searchEngine = id;
1450  }
```

```

export function setClockStyle(style: ClockStyle): void {
  settings.clockStyle = style;
}

export function setShowDate(show: boolean): void {
  settings.showDate = show;
}

export function setShowWeekday(show: boolean): void {
  settings.showWeekday = show;
}

export function setShowRecentSites(show: boolean): void {
  settings.showRecentSites = show;
}

export function setRecentSitesCount(count: number): void {
  settings.recentSitesCount = count;
}

export function setOpenInNewTab(open: boolean): void {
  settings.openInNewTab = open;
}
import type { SearchEngine } from '../types';
import { generateId } from '../types';

const STORAGE_KEY = 'quick-dial-subscription';

let isPro = $state(false);
let customEngines = $state<SearchEngine[]>([]);

function init() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (raw) {
      const data = JSON.parse(raw);
      // 未登录时清除旧的开发模式 Pro 残留
      if (!localStorage.getItem('quick-dial-token')) {
        isPro = false;
      } else {
        isPro = data.isPro || false;
      }
      customEngines = data.customEngines || [];
    }
  } catch { /* ignore */ }
}
init();

function persist() {
```

```
            targetGroup = existing;
          } else {
          // 超出免费分组数，归入默认分组
            targetGroup = defaultGroup;
          }

          const newItems = bg.items
            .filter(b => !existingUrls.has(b.url.toLowerCase()))
            .slice(0, 50)
            .map((b, i) => ({
              id: crypto.randomUUID(), title: b.title, url: b.url,
              icon: '', groupId: targetGroup.id, sortOrder: i, createdAt: Date.now(),
            }));

          state.items = [...state.items, ...newItems];
          totalImported += newItems.length;
        }

      if (totalImported === 0) throw new Error('书签已全部存在');

        initDials({ dials: state.items, groups: state.groups });

      message = `成功导入 ${totalImported} 个书签至 ${Math.min(groups.length, maxGroups)} 个分组`;
        if (!getIsPro() && groups.length > maxGroups) {
        message += `，超出 ${groups.length - maxGroups} 个文件夹自动归入"默认收藏"`;
        }
      message += '！';
        setTimeout(() => { message = ''; }, 5000);
      } catch (err) {
      message = err instanceof Error ? err.message : '导入失败';
      } finally {
        isProcessing = false;
      }
    };

    input.click();
  }

  function clearData() {
  if (!confirm(t('ie.clear') + '？此操作不可恢复！')) return;
    initDials({ dials: [], groups: [] });
    initRecentSites([]);
    localStorage.removeItem('speed-dial-data');
  message = '已清空数据';
    setTimeout(() => { message = ''; }, 2000);
  }
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content" bind:this={contentEl}>
```

```
      <h3 class="modal-title">{t('ie.title')}</h3>

      <div class="ie-actions">
        <button class="btn btn-primary" onclick={exportData} disabled={isProcessing}>
          <i class="fa-solid fa-file-export"></i> {t('ie.export')}
        </button>
        <button class="btn btn-secondary" onclick={importData} disabled={isProcessing}>
          <i class="fa-solid fa-file-import"></i> {t('ie.import')}
        </button>
        <button class="btn btn-secondary" onclick={importBookmarks} disabled={isProcessing}>
          <i class="fa-solid fa-bookmark"></i> {t('ie.importBookmarks')}
        </button>
      <p class="ie-hint">免费版最多 3 个分组，超出的文件夹自动归入"默认收藏"。<br/>开通 Pro 可创建无限分组。</p>
      </div>

      {#if message}
      <div class="ie-message" class:error={message.includes('失败') || message.includes('错误')} role="alert">
          {message}
        </div>
      {/if}

      <div class="ie-danger">
        <button class="btn btn-danger" onclick={clearData} disabled={isProcessing}>
          <i class="fa-solid fa-trash-can"></i> {t('ie.clear')}
        </button>
      </div>

      <div class="form-actions">
        <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
      </div>
    </div>
  </div>

  <style>
    .ie-actions {
      display: flex;
      flex-direction: column;
      gap: 10px;
      margin-bottom: 16px;
    }

    .ie-message {
      padding: 10px 14px;
      background: rgba(34, 197, 94, 0.08);
      color: #16a34a;
      border-radius: 8px;
      font-size: 13px;
      margin-bottom: 16px;
    }

```

```
  .ie-message.error {
    background: rgba(239, 68, 68, 0.08);
    color: #ef4444;
  }

  .ie-hint {
    font-size: 11px;
    color: var(--text-color, #1e293b);
    opacity: 0.45;
    line-height: 1.6;
    text-align: center;
    margin-top: -4px;
  }

  .ie-danger {
    padding-top: 16px;
    border-top: 1px solid var(--card-border, rgba(0,0,0,0.06));
    margin-bottom: 16px;
  }

  .btn-danger {
    background: transparent;
    color: #ef4444;
    border: 1px solid rgba(239, 68, 68, 0.3);
  }

  .btn-danger:hover {
    background: rgba(239, 68, 68, 0.08);
  }
</style>
<script lang="ts">
  import { getSettings, setSearchEngine, setClockStyle, setShowDate, setShowWeekday, setShowRecentSites, setRecentSitesCoun
  import { getIsPro } from '../stores/subscription.svelte';
  import { checkSubscription } from '../utils/payment';
  import { t, getLang, setLang } from '../utils/i18n.svelte';
  import type { ClockStyle } from '../types';

  interface Props {
    onclose: () => void;
    onsubscribe?: () => void;
  }

  let { onclose, onsubscribe }: Props = $props();

  let customCss = $state(localStorage.getItem('quick-dial-custom-css') || '');
  let customCssTimeout: ReturnType<typeof setTimeout> | undefined;

  function handleCustomCssChange(e: Event) {
    customCss = (e.target as HTMLTextAreaElement).value;
    // 防抖保存
```

```
    clearTimeout(customCssTimeout);
    customCssTimeout = setTimeout(() => {
      localStorage.setItem('quick-dial-custom-css', customCss);
      applyCustomCss(customCss);
    }, 400);
  }

  function applyCustomCss(css: string) {
    let styleEl = document.getElementById('qd-custom-css') as HTMLStyleElement | null;
    if (!styleEl) {
      styleEl = document.createElement('style');
      styleEl.id = 'qd-custom-css';
      document.head.appendChild(styleEl);
    }
    styleEl.textContent = css;
  }

  // 初始化时应用已保存的自定义 CSS
  $effect(() => {
    applyCustomCss(customCss);
  });

  // 订阅详细信息
  let subPlan = $state('');
  let subExpire = $state('');
  function pn(p: string) {
    return {monthly: t('pro.monthly'), yearly: t('pro.yearly'), lifetime: t('pro.lifetime'), free: 'Free'}[p] || p;
  };

  $effect(() => {
    if (getIsPro()) {
      checkSubscription().then(s => {
        subPlan = s.plan;
        subExpire = s.expireAt || '';
      });
    }
  });

  let overlayEl: HTMLDivElement | undefined = $state();
  let contentEl: HTMLDivElement | undefined = $state();

  $effect(() => {
    const o = overlayEl;
    const c = contentEl;
    if (!o) return;
    function handleClick(e: MouseEvent) {
      if (c && !c.contains(e.target as Node)) onclose();
    }
    function handleKey(e: KeyboardEvent) {
      if (e.key === 'Escape') onclose();
```

```
    }
    o.addEventListener('click', handleClick);
    document.addEventListener('keydown', handleKey);
    return () => {
      o.removeEventListener('click', handleClick);
      document.removeEventListener('keydown', handleKey);
    };
  });

  function handleSearchEngineChange(e: Event) {
    const select = e.target as HTMLSelectElement;
    setSearchEngine(select.value);
  }

  function handleClockStyleChange(e: Event) {
    const select = e.target as HTMLSelectElement;
    setClockStyle(select.value as ClockStyle);
  }
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content" bind:this={contentEl}>
    <h3 class="modal-title">{t('settings.title')}</h3>

    <div class="settings-list">
    <!-- 主题 -->
      <div class="setting-item">
        <div class="setting-info">
          <span class="setting-label">{t('settings.theme')}</span>
          <span class="setting-hint">{t('settings.themeHint')}</span>
        </div>
      </div>

    <!-- 搜索引擎 -->
      <div class="setting-item">
        <label class="setting-label" for="search-engine">{t('settings.engine')}</label>
        <select id="search-engine" class="form-select" value={getSettings().searchEngine} onchange={handleSearchEngineChang
          <option value="google">Google</option>
        <option value="baidu">百度</option>
          <option value="bing">Bing</option>
        <option value="sogou">搜狗</option>
        <option value="so">360搜索</option>
        <option value="zhihu">知乎</option>
        </select>
      </div>

    <!-- 时钟样式 -->
      <div class="setting-item">
        <label class="setting-label" for="clock-style">{t('clock.styles')}</label>
        <select id="clock-style" class="form-select" value={getSettings().clockStyle} onchange={handleClockStyleChange}>
```

```
            <option value="digital">{t('clock.digital')}</option>
            <option value="minimal">{t('clock.minimal')}</option>
            <option value="classic">{t('clock.classic')}</option>
            <option value="flip">{t('clock.flip')}</option>
            <option value="neon">{t('clock.neon')}</option>
            <option value="binary">{t('clock.binary')}</option>
          </select>
        </div>

      <!-- 语言 -->
        <div class="setting-item">
        <label class="setting-label" for="lang">Language / 语言</label>
          <select id="lang" class="form-select" value={getLang()} onchange={(e) => { setLang((e.target as HTMLSelectElement).
          <option value="zh-CN">中文</option>
            <option value="en">English</option>
          </select>
        </div>

      <!-- 显示日期 -->
        <div class="setting-item">
          <label class="setting-label" for="show-date">{t('settings.showDate')}</label>
          <label class="toggle">
            <input id="show-date" type="checkbox" checked={getSettings().showDate} onchange={(e) => setShowDate((e.target as
            <span class="toggle-slider"></span>
          </label>
        </div>

      <!-- 显示星期 -->
        <div class="setting-item">
          <label class="setting-label" for="show-weekday">{t('settings.showWeekday')}</label>
          <label class="toggle">
            <input id="show-weekday" type="checkbox" checked={getSettings().showWeekday} onchange={(e) => setShowWeekday((e.t
            <span class="toggle-slider"></span>
          </label>
        </div>

      <!-- 显示最近访问 -->
        <div class="setting-item">
          <label class="setting-label" for="show-recent">{t('settings.showRecent')}</label>
          <label class="toggle">
            <input id="show-recent" type="checkbox" checked={getSettings().showRecentSites} onchange={(e) => setShowRecentSit
            <span class="toggle-slider"></span>
          </label>
        </div>

      <!-- 最近访问数量 -->
        {#if getSettings().showRecentSites}
          <div class="setting-item">
            <label class="setting-label" for="recent-count">{t('settings.recentCount')}</label>
            <input
```

```
            id="recent-count"
            class="form-input"
            type="number"
            min="3"
            max="20"
            value={getSettings().recentSitesCount}
            onchange={(e) => setRecentSitesCount(parseInt((e.target as HTMLInputElement).value) || 10)}
            style="width: 80px;"
          />
        </div>
      {/if}

    <!-- 新标签页打开 -->
      <div class="setting-item">
        <label class="setting-label" for="open-new-tab">{t('settings.newTab')}</label>
        <label class="toggle">
          <input id="open-new-tab" type="checkbox" checked={getSettings().openInNewTab} onchange={(e) => setOpenInNewTab((e
          <span class="toggle-slider"></span>
        </label>
      </div>
    </div>

  <!-- 自定义 CSS（独立卡片） -->
    {#if getIsPro()}
      <div class="card-section custom-css-card">
        <div class="card-header">
         <span class="card-icon"></span>
          <span class="card-title">{t('pro.customCss')}</span>
        </div>
        <p class="card-desc">{t('pro.customCssDesc')}</p>
        <textarea id="custom-css" class="custom-css-input"
          rows="6"
          value={customCss}
          oninput={handleCustomCssChange}
          placeholder={t('pro.cssPlaceholder')}
        ></textarea>
      </div>
    {/if}

  <!-- Pro 订阅 -->
    <div class="pro-section">
      <div class="pro-header">
        <span class="pro-title">{t('pro.title')}</span>
        {#if getIsPro()}
          <span class="pro-status active">{t('pro.active')}</span>
        {:else}
          <span class="pro-status">{t('pro.inactive')}</span>
        {/if}
      </div>
```

```
7613      <!-- Pro 已激活：订阅信息 + 续费 -->
7614        {#if getIsPro()}
7615          <div class="sub-info">
7616            <div class="sub-plan-name">{pn(subPlan)}</div>
7617            {#if subExpire}
7618              <div class="sub-expire">{t('pro.expire')}{subExpire.slice(0, 10)}</div>
7619            {:else}
7620              <div class="sub-expire lifetime">终身有效</div>
7621            {/if}
7622            {#if subPlan !== 'lifetime' && onsubscribe}
7623              <button class="btn btn-outline btn-renew" onclick={onsubscribe}>{t('pro.renew')}</button>
7624            {/if}
7625          </div>
7626
7627        <!-- 自定义底部文案 -->
7628          <div class="card-section">
7629            <div class="card-header">
7630              <span class="card-icon"></span>
7631              <span class="card-title">{t('pro.customFooter')}</span>
7632            </div>
7633            <p class="card-desc">{t('pro.customFooterDesc')}</p>
7634            <div class="form-group" style="margin:0">
7635              <input class="form-input" id="custom-footer" type="text"
7636                value={localStorage.getItem('quick-dial-custom-footer') || ''}
7637                oninput={(e) => { localStorage.setItem('quick-dial-custom-footer', (e.target as HTMLInputElement).value); }}
7638                placeholder={t('pro.customFooterEg')}
7639                maxlength="40"
7640              />
7641            </div>
7642          </div>
7643        {:else}
7644          <div class="pro-features">
7645            <div class="pro-feature">
7646              <span class="feature-check"></span>
7647              <span>{t('pro.feature1')}</span>
7648            </div>
7649            <div class="pro-feature">
7650              <span class="feature-check"></span>
7651              <span>云端数据同步</span>
7652            </div>
7653            <div class="pro-feature">
7654              <span class="feature-check"></span>
7655              <span>自定义上传壁纸</span>
7656            </div>
7657            <div class="pro-feature">
7658              <span class="feature-check"></span>
7659              <span>{t('pro.feature4')}</span>
7660            </div>
7661          </div>
7662
```

```
          <div class="pro-pricing">
            <div class="pricing-option">
              <span class="pricing-price">¥9.9</span>
              <span class="pricing-period">/月</span>
            </div>
            <div class="pricing-option recommended">
              <span class="pricing-badge">推荐</span>
              <span class="pricing-price">¥68</span>
              <span class="pricing-period">/年</span>
            </div>
            <div class="pricing-option">
              <span class="pricing-price">¥198</span>
              <span class="pricing-period">终身</span>
            </div>
          </div>

          {#if onsubscribe}
            <button class="btn btn-primary btn-subscribe" onclick={onsubscribe}>
              升级 Quick Dial Pro
            </button>
          {/if}
        {/if}

      </div>

      <div class="form-actions">
        <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
      </div>
    </div>
  </div>

<style>
  .settings-list {
    display: flex;
    flex-direction: column;
    gap: 12px;
  }

  .setting-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 12px 0;
    border-bottom: 1px solid var(--card-border, rgba(0,0,0,0.04));
  }

  .setting-item:last-child {
    border-bottom: none;
  }

```

```
  .setting-label {
    font-size: 14px;
    color: var(--text-color, #1e293b);
    margin: 0;
  }

  .setting-info {
    display: flex;
    justify-content: space-between;
    align-items: center;
    width: 100%;
  }

  .setting-hint {
    font-size: 12px;
    color: var(--text-color, #1e293b);
    opacity: 0.4;
  }

  /* Toggle Switch */
  .toggle {
    position: relative;
    display: inline-block;
    width: 44px;
    height: 24px;
    cursor: pointer;
  }

  .toggle input {
    opacity: 0;
    width: 0;
    height: 0;
  }

  .toggle-slider {
    position: absolute;
    inset: 0;
    background: rgba(0,0,0,0.1);
    border-radius: 24px;
    transition: background 0.2s;
  }

  .toggle-slider::before {
    content: '';
    position: absolute;
    width: 20px;
    height: 20px;
    left: 2px;
    bottom: 2px;
    background: white;
```

```
    border-radius: 50%;
    transition: transform 0.2s;
    box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  }

  .toggle input:checked + .toggle-slider {
    background: var(--primary-color, #4f46e5);
  }

  .toggle input:checked + .toggle-slider::before {
    transform: translateX(20px);
  }

  .form-actions {
    display: flex;
    justify-content: flex-end;
    margin-top: 16px;
  }

  /* Pro Section */
  .pro-section {
    margin-top: 20px;
    padding: 16px;
    border: 1px solid var(--card-border, rgba(0,0,0,0.08));
    border-radius: 14px;
    background: linear-gradient(135deg, rgba(99, 102, 241, 0.04), rgba(168, 85, 247, 0.04));
  }

  .pro-header {
    display: flex;
    align-items: center;
    gap: 8px;
    margin-bottom: 12px;
  }

  .pro-title {
    font-size: 15px;
    font-weight: 700;
    color: var(--text-color, #1e293b);
    flex: 1;
  }

  .pro-status {
    font-size: 11px;
    padding: 2px 8px;
    border-radius: 10px;
    background: rgba(0,0,0,0.06);
    color: var(--text-color, #1e293b);
    opacity: 0.5;
  }
```

```
7813
7814    .pro-status.active {
7815      background: linear-gradient(135deg, rgba(59, 130, 246, 0.15), rgba(37, 99, 235, 0.15));
7816      color: #3b82f6;
7817      opacity: 1;
7818    }
7819
7820    .pro-features {
7821      display: flex;
7822      flex-direction: column;
7823      gap: 6px;
7824      margin-bottom: 12px;
7825    }
7826
7827    .pro-feature {
7828      display: flex;
7829      align-items: center;
7830      gap: 8px;
7831      font-size: 12px;
7832      color: var(--text-color, #1e293b);
7833      opacity: 0.7;
7834    }
7835
7836    .feature-check {
7837      color: #22c55e;
7838      font-weight: 700;
7839    }
7840
7841    .pro-pricing {
7842      display: flex;
7843      gap: 8px;
7844      margin-bottom: 14px;
7845    }
7846
7847    .pricing-option {
7848      flex: 1;
7849      display: flex;
7850      flex-direction: column;
7851      align-items: center;
7852      padding: 10px 8px;
7853      border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7854      border-radius: 10px;
7855      background: var(--card-bg, rgba(255,255,255,0.5));
7856      position: relative;
7857    }
7858
7859    .pricing-option.recommended {
7860      border-color: #3b82f6;
7861      background: rgba(59, 130, 246, 0.06);
7862    }
```

```
7863
7864    .pricing-badge {
7865      position: absolute;
7866      top: -8px;
7867      font-size: 10px;
7868      padding: 1px 8px;
7869      border-radius: 8px;
7870      background: linear-gradient(135deg, #3b82f6, #1d4ed8);
7871      color: white;
7872      font-weight: 600;
7873    }
7874
7875    .pricing-price {
7876      font-size: 18px;
7877      font-weight: 700;
7878      color: var(--text-color, #1e293b);
7879      line-height: 1;
7880    }
7881
7882    .pricing-period {
7883      font-size: 11px;
7884      color: var(--text-color, #1e293b);
7885      opacity: 0.5;
7886      margin-top: 2px;
7887    }
7888
7889    /* 自定义 CSS 独立卡片 */
7890    .card-section {
7891      border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7892      border-radius: 14px;
7893      padding: 18px;
7894      margin-bottom: 16px;
7895      background: var(--card-bg, rgba(255,255,255,0.5));
7896    }
7897    .card-header { display: flex; align-items: center; gap: 8px; margin-bottom: 6px; }
7898    .card-icon { font-size: 16px; }
7899    .card-title { font-size: 14px; font-weight: 700; color: var(--text-color); }
7900    .card-desc { font-size: 12px; color: var(--text-color); opacity: 0.4; margin-bottom: 12px; }
7901
7902    .custom-css-input {
7903      width: 100%;
7904      padding: 12px;
7905      border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7906      border-radius: 10px;
7907      font-family: 'Fira Code', 'Consolas', monospace;
7908      font-size: 13px;
7909      line-height: 1.6;
7910      background: var(--input-bg, rgba(0,0,0,0.02));
7911      color: var(--text-color);
7912      resize: vertical;
```

```
    outline: none;
  }
  .custom-css-input:focus { border-color: #3b82f6; }

/* 订阅信息 */
  .sub-info {
    background: var(--hover-bg, rgba(0,0,0,0.02));
    border-radius: 12px;
    padding: 16px;
    margin-bottom: 12px;
  }
  .sub-plan-name { font-size: 16px; font-weight: 700; color: var(--text-color); margin-bottom: 4px; }
  .sub-expire { font-size: 13px; color: var(--text-color); opacity: 0.5; margin-bottom: 10px; }
  .sub-expire.lifetime { color: #a855f7; opacity: 1; font-weight: 600; }
  .btn-renew { font-size: 12px; padding: 8px 16px; }

  .btn-subscribe {
    width: 100%;
    margin-top: 12px;
    padding: 12px;
    font-size: 15px;
    font-weight: 600;
    background: linear-gradient(135deg, #3b82f6, #1d4ed8);
  }
</style>
<script lang="ts">
import { t } from '../utils/i18n.svelte';
  import { getClickCounts, getTotalClicks, clearClickCounts } from '../stores/recentSites.svelte';

  interface Props {
    onclose: () => void;
  }

  let { onclose }: Props = $props();

  let overlayEl: HTMLDivElement | undefined = $state();
  let contentEl: HTMLDivElement | undefined = $state();

  $effect(() => {
    const o = overlayEl;
    const c = contentEl;
    if (!o) return;
    function handleClick(e: MouseEvent) {
      if (c && !c.contains(e.target as Node)) onclose();
    }
    function handleKey(e: KeyboardEvent) {
      if (e.key === 'Escape') onclose();
    }
    o.addEventListener('click', handleClick);
    document.addEventListener('keydown', handleKey);
```

```
    return () => {
      o.removeEventListener('click', handleClick);
      document.removeEventListener('keydown', handleKey);
    };
  });

  function getTopSites(): { url: string; count: number }[] {
    const counts = getClickCounts();
    return Object.entries(counts)
      .sort(([, a], [, b]) => b - a)
      .slice(0, 10)
      .map(([url, count]) => ({ url, count }));
  }

  function formatUrl(url: string): string {
    try {
      return new URL(url).hostname;
    } catch {
      return url;
    }
  }

  function exportCSV() {
    const counts = getClickCounts();
    const rows = Object.entries(counts).sort(([,a],[,b]) => b - a);
    let csv = '网站,点击次数\n';
    for (const [url, count] of rows) {
      csv += `"${url}",${count}\n`;
    }
    const blob = new Blob(['\uFEFF' + csv], { type: 'text/csv;charset=utf-8' });
    const a = document.createElement('a');
    a.href = URL.createObjectURL(blob);
    a.download = 'quick-dial-stats-' + new Date().toISOString().slice(0, 10) + '.csv';
    a.click();
    URL.revokeObjectURL(a.href);
  }
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content" bind:this={contentEl}>
    <h3 class="modal-title">{t('stats.title')}</h3>

    <div class="stats-summary">
      <div class="stat-item">
        <span class="stat-value">{getTotalClicks()}</span>
        <span class="stat-label">{t('stats.total')}</span>
      </div>
      <div class="stat-item">
        <span class="stat-value">{Object.keys(getClickCounts()).length}</span>
        <span class="stat-label">{t('stats.sites')}</span>
```

```
      </div>
    </div>

    {#if getTopSites().length > 0}
      <div class="stats-list">
        <h4 class="stats-subtitle">{t('stats.ranking')}</h4>
        {#each getTopSites() as site, i}
          <div class="stats-row">
            <span class="stats-rank">{i + 1}</span>
            <span class="stats-url">{formatUrl(site.url)}</span>
            <span class="stats-count">{site.count} 次</span>
          </div>
        {/each}
      </div>
    {:else}
      <p class="stats-empty">{t('stats.empty')}</p>
    {/if}

    <div class="form-actions">
      <button class="btn btn-secondary" onclick={exportCSV}>{t('stats.export')}</button>
      <button class="btn btn-secondary" onclick={() => { clearClickCounts(); }}>{t('stats.clear')}</button>
      <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
    </div>
  </div>
</div>

<style>
  .stats-summary {
    display: flex;
    gap: 16px;
    margin-bottom: 16px;
  }
  .stat-item {
    flex: 1;
    text-align: center;
    padding: 12px;
    background: var(--hover-bg, rgba(0,0,0,0.04));
    border-radius: 10px;
  }
  .stat-value {
    display: block;
    font-size: 24px;
    font-weight: 700;
    color: var(--primary-color, #3b82f6);
  }
  .stat-label {
    font-size: 12px;
    color: var(--text-color, #1e293b);
    opacity: 0.5;
  }
```

```
  .stats-list { margin-top: 8px; }
  .stats-subtitle { font-size: 13px; margin-bottom: 8px; opacity: 0.5; }
  .stats-row {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 6px 0;
    border-bottom: 1px solid var(--card-border, rgba(0,0,0,0.04));
  }
  .stats-rank { width: 20px; font-size: 12px; font-weight: 600; opacity: 0.4; text-align: center; }
  .stats-url { flex: 1; font-size: 13px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
  .stats-count { font-size: 12px; opacity: 0.5; }
  .stats-empty { text-align: center; padding: 20px; opacity: 0.4; font-size: 13px; }
  .form-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 16px; }
</style>
<script lang="ts">
import { t } from '../utils/i18n.svelte';
  import { getShortcuts } from '../utils/keyboard';

  interface Props {
    onclose: () => void;
  }

  let { onclose }: Props = $props();

  let overlayEl: HTMLDivElement | undefined = $state();
  let contentEl: HTMLDivElement | undefined = $state();
  const shortcuts = getShortcuts();

  $effect(() => {
    const o = overlayEl;
    const c = contentEl;
    if (!o) return;
    function handleClick(e: MouseEvent) {
      if (c && !c.contains(e.target as Node)) onclose();
    }
    function handleKey(e: KeyboardEvent) {
      if (e.key === 'Escape' || e.key === '?') onclose();
    }
    o.addEventListener('click', handleClick);
    document.addEventListener('keydown', handleKey);
    return () => {
      o.removeEventListener('click', handleClick);
      document.removeEventListener('keydown', handleKey);
    };
  });
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content" bind:this={contentEl}>
```

```
    <h3 class="modal-title"> {t('help.title')}</h3>
      <div class="shortcut-list">
        {#each shortcuts as s}
          <div class="shortcut-row">
            <kbd class="shortcut-key">{s.key}</kbd>
            <span class="shortcut-desc">{s.description}</span>
          </div>
        {/each}
      </div>
    <p class="help-tip">按 <kbd>?</kbd> 随时打开此面板</p>
  </div>
</div>

<style>
  .shortcut-list { display: flex; flex-direction: column; gap: 10px; margin-bottom: 16px; }
  .shortcut-row { display: flex; align-items: center; gap: 12px; }
  .shortcut-key {
    display: inline-block;
    padding: 4px 10px;
    font-size: 12px;
    font-family: monospace;
    font-weight: 600;
    background: var(--hover-bg, rgba(0,0,0,0.04));
    border: 1px solid var(--card-border);
    border-radius: 6px;
    color: var(--text-color);
    min-width: 120px;
    text-align: center;
  }
  .shortcut-desc { font-size: 14px; color: var(--text-color); opacity: 0.7; }
  .help-tip { text-align: center; font-size: 12px; opacity: 0.35; margin-top: 8px; }
</style>
<script lang="ts">
import { t } from '../utils/i18n.svelte';
  import { onMount } from 'svelte';

  interface Step {
    title: string;
    desc: string;
    icon: string;
  }

  const steps: Step[] = [
    { title: t('onboard.step1Title'), desc: '在搜索框输入关键词，按 Enter 或点搜索按钮', icon: '' },
    { title: t('onboard.step2Title'), desc: '点右下角 + 按钮，粘贴网址自动识别名称和图标', icon: '' },
    { title: t('onboard.step3Title'), desc: '点  按钮选择 12 种精美渐变，打造专属风格', icon: '' },
    { title: t('onboard.step4Title'), desc: '前往官网 www.cilacila.cn 绑定邮箱、找回密码、管理订阅', icon: '' },
  ];

  let show = $state(false);
```

```
  let current = $state(0);
  let dismissed = $state(false);

  onMount(() => {
    if (localStorage.getItem('qd-onboarded')) return;
    setTimeout(() => show = true, 500);
  });

  function next() {
    if (current < steps.length - 1) {
      current++;
    } else {
      dismiss();
    }
  }

  function dismiss() {
    show = false;
    dismissed = true;
    localStorage.setItem('qd-onboarded', '1');
  }

  function dotClick(i: number) {
    current = i;
  }
</script>

{#if show && !dismissed}
  <div class="onboard-overlay">
    <div class="onboard-card">
      <span class="onboard-icon">{steps[current].icon}</span>
      <h2 class="onboard-title">{steps[current].title}</h2>
      <p class="onboard-desc">{steps[current].desc}</p>

      <div class="onboard-dots">
        {#each steps as _, i}
<button class="onboard-dot" class:active={current === i} onclick={() => dotClick(i)} aria-label="第{i + 1}步"></but
        {/each}
      </div>

      <div class="onboard-actions">
        <button class="btn btn-secondary" onclick={dismiss}>{t('onboard.skip')}</button>
        <button class="btn btn-primary" onclick={next}>
          {current < steps.length - 1 ? t('onboard.next') : t('onboard.start')}
        </button>
      </div>
    </div>
  </div>
{/if}

```

```
<style>
  .onboard-overlay {
    position: fixed;
    inset: 0;
    z-index: 9999;
    background: rgba(0,0,0,0.6);
    display: flex;
    align-items: center;
    justify-content: center;
    animation: fadeIn 0.3s ease;
  }
  .onboard-card {
    background: var(--card-bg, #fff);
    border-radius: 20px;
    padding: 40px 32px 28px;
    text-align: center;
    max-width: 340px;
    width: 90%;
    animation: slideUp 0.3s ease;
  }
  .onboard-icon { font-size: 48px; display: block; margin-bottom: 16px; }
  .onboard-title { font-size: 22px; font-weight: 700; color: var(--text-color); margin-bottom: 8px; }
  .onboard-desc { font-size: 14px; color: var(--text-color); opacity: 0.6; line-height: 1.6; margin-bottom: 24px; }
  .onboard-dots { display: flex; justify-content: center; gap: 8px; margin-bottom: 20px; }
  .onboard-dot {
    width: 8px; height: 8px; border-radius: 50%; border: none;
    background: var(--card-border); cursor: pointer; padding: 0; transition: all 0.2s;
  }
  .onboard-dot.active { background: #3b82f6; width: 24px; border-radius: 4px; }
  .onboard-actions { display: flex; justify-content: space-between; gap: 12px; }
  .onboard-actions .btn { flex: 1; padding: 12px; font-size: 14px; }

  @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
  @keyframes slideUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }
</style>
<script lang="ts">
import { t } from '../utils/i18n.svelte';
  import { getUsername } from '../utils/sync';
  import { PLANS, createOrder, generateQRCodeSVG } from '../utils/payment';
  import type { PlanInfo } from '../utils/payment';

  interface Props {
    onclose: () => void;
  }

  let { onclose }: Props = $props();

  let selectedPlan = $state('yearly');
  let payMethod = $state<'wechat' | 'alipay'>('wechat');
  let status = $state('');
```

```
  let statusOk = $state(true);
  let loading = $state(false);
  let qrCode = $state('');
  let orderInfo = $state<{ plan: string; amount: number } | null>(null);
  let showQr = $state(false);

  let overlayEl: HTMLDivElement | undefined = $state();
  let contentEl: HTMLDivElement | undefined = $state();

  $effect(() => {
    const o = overlayEl;
    const c = contentEl;
    if (!o) return;
    function handleClick(e: MouseEvent) {
      if (c && !c.contains(e.target as Node)) onclose();
    }
    function handleKey(e: KeyboardEvent) {
      if (e.key === 'Escape') onclose();
    }
    o.addEventListener('click', handleClick);
    document.addEventListener('keydown', handleKey);
    return () => {
      o.removeEventListener('click', handleClick);
      document.removeEventListener('keydown', handleKey);
    };
  });

  async function handlePay() {
    loading = true;
    status = '';
    qrCode = '';
    showQr = false;

    const result = await createOrder(selectedPlan, payMethod);
    if (result.ok) {
      if (result.qrCode) {
        qrCode = generateQRCodeSVG(result.qrCode);
        orderInfo = { plan: result.plan || selectedPlan, amount: result.amount || 0 };
        showQr = true;
      status = '请使用' + (payMethod === 'wechat' ? '微信' : '支付宝') + '扫码支付';
        statusOk = true;
      } else if (result.payUrl) {
      // 支付宝 page_pay 兜底
        window.open(result.payUrl, '_blank');
      status = '已在新窗口打开支付页面';
        statusOk = true;
      } else {
      status = '获取支付链接失败';
        statusOk = false;
      }
```

```
    } else {
      status = result.msg;
      statusOk = false;
    }
    loading = false;
  }

  function getPlanPrice(plan: PlanInfo): string {
  return '¥' + plan.amount.toFixed(plan.amount === Math.floor(plan.amount) ? 0 : 2);
  }
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content subscribe-modal" bind:this={contentEl}>
  <h3 class="modal-title">{t('sub.title')}</h3>

    {#if showQr}
      ...
    <!-- 支付二维码 -->
      <div class="qr-section">
      <p class="qr-hint">使用 {payMethod === 'wechat' ? '微信' : '支付宝'} 扫码支付</p>
        {#if qrCode}
        <img class="qr-img" src={qrCode} alt="支付二维码" />
        {/if}
        {#if orderInfo}
          <div class="qr-info">
          <span>{PLANS.find(p => p.id === orderInfo.plan)?.name}套餐</span>
          <span class="qr-amount">¥{orderInfo.amount.toFixed(2)}</span>
          </div>
        {/if}
        <p class="qr-tip">{t('sub.paidRefresh')}</p>
      </div>
      <div class="form-actions">
      <button class="btn btn-secondary" onclick={() => showQr = false}>返回</button>
        <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
      </div>
    {:else}
    <!-- 选择套餐 -->
    <p class="subscribe-user">{getUsername()}</p>

      <div class="plan-list">
        {#each PLANS as plan}
          <label class="plan-card" class:selected={selectedPlan === plan.id}>
            <input type="radio" name="plan" value={plan.id} bind:group={selectedPlan} hidden />
            {#if plan.id === 'yearly'}
            <span class="plan-badge">推荐</span>
            {/if}
            <span class="plan-name">{plan.name}</span>
            <span class="plan-price">{getPlanPrice(plan)}</span>
          <span class="plan-period">{plan.days ? `/${plan.days}天` : '永久'}</span>
```

```
            </label>
          {/each}
        </div>

    <!-- 支付方式 -->
        <div class="pay-methods">
          <label class="pay-method" class:selected={payMethod === 'wechat'}>
            <input type="radio" name="method" value="wechat" bind:group={payMethod} hidden />
          <span class="pay-icon"></span>
          <span>微信支付</span>
          </label>
          <label class="pay-method" class:selected={payMethod === 'alipay'}>
            <input type="radio" name="method" value="alipay" bind:group={payMethod} hidden />
          <span class="pay-icon"></span>
          <span>支付宝</span>
          </label>
        </div>

        <div class="form-actions">
          <button class="btn btn-primary btn-pay" onclick={handlePay} disabled={loading}>
            {loading ? t('sub.creating') : t('sub.pay')}
          </button>
          <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
        </div>
      {/if}

      {#if status}
        <p class="pay-status" class:ok={statusOk} class:error={!statusOk}>{status}</p>
      {/if}
    </div>
  </div>

<style>
  .subscribe-modal { max-width: 420px; }

  .subscribe-user {
    font-size: 14px;
    margin-bottom: 12px;
    opacity: 0.7;
  }

  .plan-list {
    display: flex;
    gap: 8px;
    margin-bottom: 16px;
  }

  .plan-card {
    flex: 1;
    display: flex;
```

```
    flex-direction: column;
    align-items: center;
    padding: 14px 8px;
    border: 2px solid var(--card-border, rgba(0,0,0,0.08));
    border-radius: 12px;
    cursor: pointer;
    position: relative;
    transition: border-color 0.2s;
    background: var(--card-bg, rgba(255,255,255,0.5));
  }

  .plan-card.selected {
    border-color: #3b82f6;
    background: rgba(59, 130, 246, 0.04);
  }

  .plan-badge {
    position: absolute;
    top: -10px;
    font-size: 10px;
    padding: 2px 8px;
    border-radius: 8px;
    background: linear-gradient(135deg, #3b82f6, #1d4ed8);
    color: white;
    font-weight: 600;
  }

  .plan-name {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-color);
    margin-bottom: 4px;
  }

  .plan-price {
    font-size: 22px;
    font-weight: 700;
    color: var(--text-color);
    line-height: 1;
  }

  .plan-period {
    font-size: 11px;
    color: var(--text-color);
    opacity: 0.4;
    margin-top: 2px;
  }

  .pay-methods {
    display: flex;
```

```
    gap: 8px;
    margin-bottom: 16px;
  }

  .pay-method {
    flex: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 6px;
    padding: 12px;
    border: 2px solid var(--card-border, rgba(0,0,0,0.08));
    border-radius: 10px;
    cursor: pointer;
    font-size: 14px;
    transition: border-color 0.2s;
  }

  .pay-method.selected {
    border-color: #3b82f6;
  }

  .pay-icon { font-size: 16px; }

  .btn-pay {
    padding: 12px 24px;
    font-size: 15px;
    font-weight: 600;
  }

  .qr-section {
    text-align: center;
    padding: 16px 0;
  }

  .qr-hint {
    font-size: 14px;
    margin-bottom: 12px;
    color: var(--text-color);
    opacity: 0.7;
  }

  .qr-img {
    width: 200px;
    height: 200px;
    border-radius: 12px;
    border: 2px solid var(--card-border);
    background: white;
    padding: 8px;
  }
```

```
8513
8514    .qr-info {
8515      display: flex;
8516      justify-content: center;
8517      align-items: center;
8518      gap: 12px;
8519      margin-top: 12px;
8520      font-size: 14px;
8521      color: var(--text-color);
8522    }
8523
8524    .qr-amount {
8525      font-size: 20px;
8526      font-weight: 700;
8527      color: #ef4444;
8528    }
8529
8530    .qr-tip {
8531      font-size: 12px;
8532      color: var(--text-color);
8533      opacity: 0.4;
8534      margin-top: 12px;
8535    }
8536
8537    .pay-status {
8538      font-size: 12px;
8539      padding: 8px 12px;
8540      border-radius: 8px;
8541      margin-top: 8px;
8542    }
8543
8544    .pay-status.ok {
8545      background: rgba(34, 197, 94, 0.1);
8546      color: #16a34a;
8547    }
8548
8549    .pay-status.error {
8550      background: rgba(239, 68, 68, 0.08);
8551      color: #ef4444;
8552    }
8553
8554    .form-actions {
8555      display: flex;
8556      justify-content: flex-end;
8557      gap: 8px;
8558      margin-top: 12px;
8559    }
8560  </style>
8561  <script lang="ts">
8562  import { t } from '../utils/i18n.svelte';
```

```
8563  import { getIsPro, syncProStatus } from '../stores/subscription.svelte';
8564  import { isLoggedIn, getUsername, login, register, logout, uploadSync, downloadSync, getLastSyncTime } from '../utils/syn
8565  import { loadData, saveData } from '../utils/storage';
8566  import { showToast } from '../utils/toast.svelte';
8567
8568  interface Props {
8569    onclose: () => void;
8570  }
8571
8572  let { onclose }: Props = $props();
8573
8574  let username = $state('');
8575  let password = $state('');
8576  let isLogin = $state(true);
8577  let status = $state('');
8578  let statusOk = $state(true);
8579  let loading = $state(false);
8580
8581  let overlayEl: HTMLDivElement | undefined = $state();
8582  let contentEl: HTMLDivElement | undefined = $state();
8583
8584  $effect(() => {
8585    const o = overlayEl;
8586    const c = contentEl;
8587    if (!o) return;
8588    function handleClick(e: MouseEvent) {
8589      if (c && !c.contains(e.target as Node)) onclose();
8590    }
8591    function handleKey(e: KeyboardEvent) {
8592      if (e.key === 'Escape') onclose();
8593    }
8594    o.addEventListener('click', handleClick);
8595    document.addEventListener('keydown', handleKey);
8596    return () => {
8597      o.removeEventListener('click', handleClick);
8598      document.removeEventListener('keydown', handleKey);
8599    };
8600  });
8601
8602  async function handleAuth() {
8603    if (!username.trim() || !password.trim()) {
8604      status = '请填写' + t('sync.user') + '和' + t('sync.password');
8605      statusOk = false;
8606      return;
8607    }
8608    loading = true;
8609    status = '';
8610    const result = isLogin ? await login(username.trim(), password) : await register(username.trim(), password);
8611    if (result.ok) {
8612      syncProStatus();
```

```
      showToast(isLogin ? t('sync.login') + '成功' : t('sync.register') + '成功', 'success');
        onclose();
        return;
      }
      loading = false;
      status = result.msg;
      statusOk = result.ok;
    }

    async function handleUpload() {
      loading = true;
      status = '';
      const result = await uploadSync();
      loading = false;
      status = result.msg;
      statusOk = result.ok;
    }

    async function handleDownload() {
      loading = true;
      status = '';
      const result = await downloadSync();
      if (result.ok && result.data) {
        saveData(result.data as any);
      // 同步自定义 CSS
        if (result.data.customCss !== undefined) {
          localStorage.setItem('quick-dial-custom-css', result.data.customCss);
        // 注入到页面
          let styleEl = document.getElementById('qd-custom-css') as HTMLStyleElement | null;
          if (!styleEl) {
            styleEl = document.createElement('style');
            styleEl.id = 'qd-custom-css';
            document.head.appendChild(styleEl);
          }
          styleEl.textContent = result.data.customCss;
        }
        status = t('sync.downloaded');
        statusOk = true;
        setTimeout(() => window.location.reload(), 3000);
      } else {
        status = result.msg;
        statusOk = false;
      }
      loading = false;
    }

    function handleLogout() {
      logout();
      showToast(t('sync.logout'), 'info');
      onclose();
```

```
  }

  function formatTime(iso: string | null): string {
    if (!iso) return t('sync.never');
    try {
      const d = new Date(iso);
      return d.toLocaleString('zh-CN');
    } catch { return iso; }
  }
</script>

<div class="modal-overlay" bind:this={overlayEl}>
  <div class="modal-content" bind:this={contentEl}>
    <h3 class="modal-title"> {t('sync.title')}</h3>

    {#if isLoggedIn()}
      <!-- 已{t('sync.login')} -->
      <p class="sync-notice"> {t('sync.manual')}</p>
      <div class="sync-status">
        <span class="sync-user"> {getUsername()}</span>
        <span class="sync-time">{t('sync.syncTime')}: {formatTime(getLastSyncTime())}</span>
      </div>

      {#if getIsPro()}
        <div class="sync-actions">
          <button class="btn btn-primary" onclick={handleUpload} disabled={loading}>
           {loading ? '同步中...' : ' ' + t('sync.upload')}
          </button>
          <button class="btn btn-secondary" onclick={handleDownload} disabled={loading}>
           {' ' + t('sync.download')}
          </button>
        </div>
      {:else}
        <p class="sync-pro-only"> 云同步需 Pro，<span class="pro-link">去设置升级</span></p>
      {/if}

      <div class="form-actions">
        <button class="btn btn-secondary" onclick={handleLogout}>{t('sync.logout')}</button>
        <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
      </div>
    {:else}
      <!-- {t('sync.login')}/{t('sync.register')} -->
      <div class="form-group">
        <label class="form-label" for="sync-username">{t('sync.user')}</label>
        <input id="sync-username" class="form-input" type="text" bind:value={username} placeholder="至少2个字符" />
      </div>
      <div class="form-group">
        <label class="form-label" for="sync-password">{t('sync.password')}</label>
        <input id="sync-password" class="form-input" type="password" bind:value={password} placeholder="至少6位" onkeydown={(e
      </div>
```

```
8713
8714        <div class="form-actions">
8715          <button class="btn btn-primary" onclick={handleAuth} disabled={loading}>
8716          {loading ? '处理中...' : (isLogin ? t('sync.login') : t('sync.register'))}
8717          </button>
8718          <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
8719        </div>
8720
8721        <p class="auth-switch">
8722          {#if isLogin}
8723          没有账号？<button class="link-btn" onclick={() => { isLogin = false; status = ''; }}>立即{t('sync.register')}</button>
8724          {:else}
8725          已有账号？<button class="link-btn" onclick={() => { isLogin = true; status = ''; }}>去{t('sync.login')}</button>
8726          {/if}
8727        </p>
8728      {/if}
8729
8730      {#if status}
8731        <p class="sync-status-msg" class:ok={statusOk} class:error={!statusOk}>
8732          {status}
8733        </p>
8734      {/if}
8735    </div>
8736  </div>
8737
8738  <style>
8739    .sync-status {
8740      display: flex;
8741      flex-direction: column;
8742      gap: 4px;
8743      margin-bottom: 16px;
8744      padding: 12px;
8745      background: var(--hover-bg);
8746      border-radius: 10px;
8747    }
8748    .sync-user { font-size: 14px; font-weight: 600; }
8749    .sync-time { font-size: 12px; opacity: 0.5; }
8750    .sync-notice { font-size: 12px; opacity: 0.5; margin-bottom: 12px; line-height: 1.6; }
8751    .sync-actions { display: flex; gap: 8px; margin-bottom: 16px; }
8752    .sync-actions .btn { flex: 1; }
8753    .sync-pro-only { text-align: center; padding: 20px; opacity: 0.5; font-size: 13px; line-height: 1.8; }
8754    .auth-switch { text-align: center; font-size: 13px; color: var(--text-color); opacity: 0.5; margin-top: 12px; }
8755    .link-btn { background: none; border: none; color: #3b82f6; font-size: 13px; font-weight: 600; cursor: pointer; padding:
8756    .sync-status-msg { font-size: 12px; padding: 8px 12px; border-radius: 8px; margin-top: 8px; }
8757    .sync-status-msg.ok { background: rgba(34, 197, 94, 0.1); color: #16a34a; }
8758    .sync-status-msg.error { background: rgba(239, 68, 68, 0.08); color: #ef4444; }
8759    .form-group { margin-bottom: 12px; }
8760    .form-label { display: block; font-size: 13px; margin-bottom: 4px; opacity: 0.7; }
8761    .form-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 12px; }
8762  </style>
```